DISTRIBUTED AI WORKSHOP OPERATIONS

Intel × Red Hat AI Launchpad

Order once. Learn in isolation. Reclaim completely.

Red Hat

The operating problem

One workshop is more than a set of pods.

- **Content** must match the deployed workload
- **25 identities** need isolated, usable environments
- **Placement** must respect real cluster capacity
- **Model and operator access** must work for participants
- **Reclaim** must remove every owned artifact

> The product is the complete order-to-zero-residue lifecycle.

Pilot proof: three workshops, 75 participants

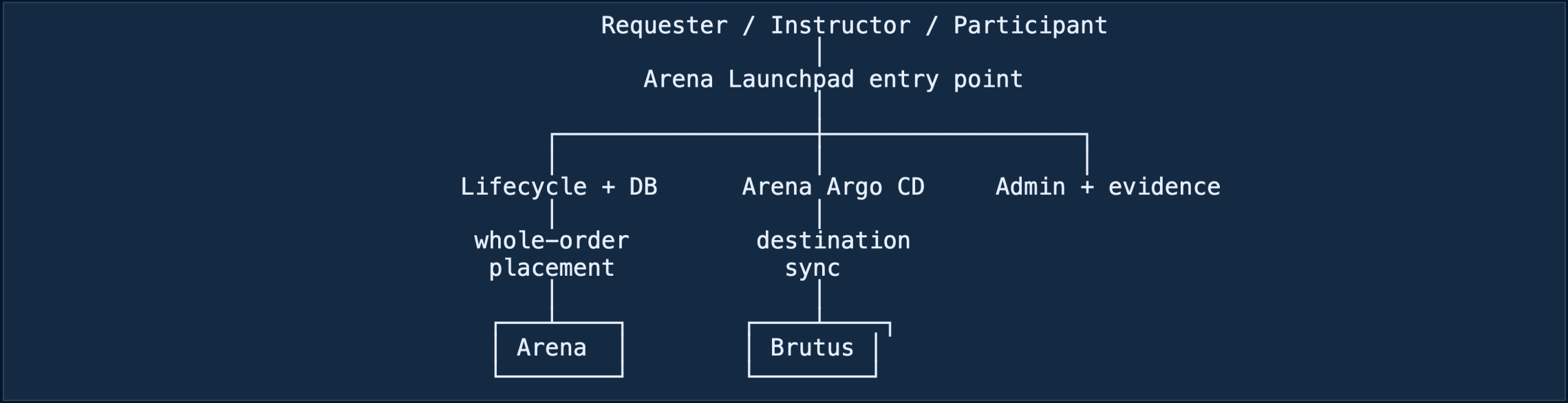
Workshop	Execution cluster	Result
Building an AI Agent	Brutus	**25 / 25**
Build Multi-Agent AI Systems	Arena	**25 / 25**
Serve LLMs on Intel Xeon	Arena	**25 / 25**

Staggered provisioning. Concurrent use. Zero-residue group reclaim.

GREEN-live automated pilot gate

Manual visual and public-access gates remain separate

One control plane, distributed execution



Oberon: excluded pending cluster remediation

Flightpath: passive DR candidate, not an active writer

The lifecycle is the contract

```
Git intent → preview → reserve → persist cluster_ref → provision
                                                          ↓
zero residue ← release ← reclaim ← use ← handoff ← validate
```

- One workshop stays on **one eligible cluster**
- Capacity is reserved for the **complete order**
- Participant behavior—not pod status—decides readiness
- Retry and cleanup stay on the persisted `cluster_ref`
- Reclaim is idempotent and ends with an explicit residue check

The participant experience

Open Lab provides one guided seat experience:

1. Version-pinned Showroom learning journey
2. Terminal already scoped to the assigned namespace
3. Operator and application tools embedded for the lab
4. Real catalog-specific agent or model exercise
5. Namespace authorization with cross-seat denial

The participant should not need to understand cluster placement to learn.

Live demo path

Minute	Show	Prove
0–4	Catalog and 25-seat request	Versioned intent and one-order model
4–7	Capacity and placement	Complete-order fit and persisted target
7–10	Admin operations	Order, cluster, seat, and failure visibility
10–15	Participant lab	Content, terminal scope, operator/app workflow
15–18	Architecture and evidence	Distributed execution with one entry point
18–20	Group reclaim	Terminal state and zero residue

Evidence is part of delivery

Every certification run records:

- commit SHA, image digests, and catalog version
- order, seat, namespace, tenant, and cluster correlation
- capacity, readiness, function, isolation, and timing results
- audit and lifecycle events
- group reclaim and zero-residue counts
- artifact hashes and a scored release rubric

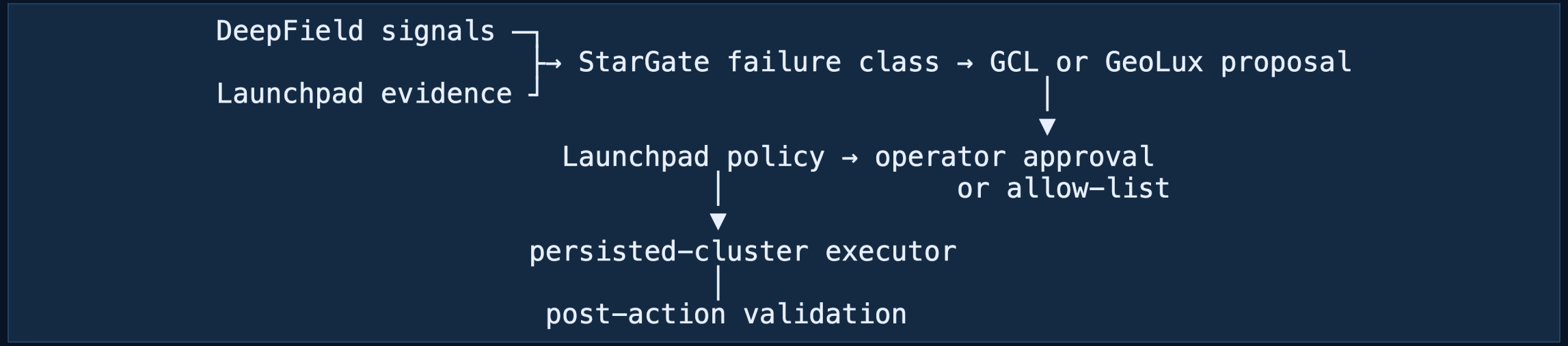
RED stays immutable. GREEN must point to proof.

Scale by measured graduation

Gate	Scale	Decision
Source and component	local	structure, security, deterministic contract
First participant	1	complete learner journey and cleanup
Concurrency	5	isolation, shared services, model behavior
Workshop	25	readiness SLO, functional load, bulk reclaim
Larger workshop	50 → 75	only after node and headroom proof
Fleet	multiple orders	queue fairness and failure-domain rehearsal

> Never publish a seat limit from allocatable capacity alone.

The operations intelligence loop



Launchpad remains the lifecycle mutation authority.

Autonomy has a safety ladder

1. **Observe** — attach evidence; operator uses the runbook
2. **Recommend** — show one bounded action and affected resources
3. **Approve** — authenticated, idempotent execution
4. **Automatic** — allow-listed, retry-budgeted, validated, circuit-broken

Start with validation retry, Showroom resync, owned Route reconciliation, managed-workload restart, expiry, reclaim retry, and stale-record repair.

RBAC, Secrets, Operators, nodes, storage, DNS, models, and cross-cluster migration always require human authority.

Onboard a quickstart without bespoke platform edits

```
immutable quickstart repo
          ↓ discover
fail-closed intake + discovery receipt
          ↓ review
catalog + Showroom + workload contract
          ↓ prove
source build → 1 seat → 5 seats → 25 seats
          ↓ approve
orderable catalog → supported use → reclaim
```

Git remains the source of truth. Promotion remains explicit.

Deploy the ecosystem to another cluster

The portable playbooks enforce a repeatable path:

- read-only API, node, storage, and credential preflight
- server-side apply of the reviewed control-plane overlay
- one least-privilege provisioner identity per execution cluster
- a separate Argo CD destination identity
- immutable image, model, ingress, and capability registration
- API readiness and cluster eligibility validation
- Launchpad-driven group reclaim and zero-residue proof

Every `oc` invocation uses an explicit kubeconfig.

Next gates

1. Complete requester, participant, lab, and admin visual acceptance
2. Establish stable public DNS, TLS, ingress, and SSO ownership
3. Route participant inference through version-pinned LiteLLM and prove attribution
4. Deploy and manually validate the Keycloak 26.7.2 candidate
5. Exercise lifecycle HA and Flightpath promotion/failback
6. Spike the GCL/GeoLux decision-provider contract
7. Onboard the next quickstart through the new repository pipeline

One entry point.

Distributed workshops. Governed operations. Verifiable cleanup.

Red Hat

github.com/rhpds/launchpad